



Full length article

PublicAgent: Multi-agent design principles from an LLM-based open data analysis framework

Sina Montazeri, Yunhe Feng, Kewei Sha *

University of North Texas, Denton, TX, USA

ARTICLE INFO

Keywords:

Multi-agent systems
Large language models
Open data
Data analysis
Natural language interfaces
Agent orchestration

ABSTRACT

Open data repositories hold potential for evidence-based decision-making, yet remain inaccessible to non-experts who lack expertise in dataset discovery, schema mapping, and statistical analysis. While large language models show promise for individual tasks, end-to-end analytical workflows expose fundamental limitations: attention dilutes across growing contexts, specialized reasoning patterns interfere, and errors propagate without detection. We present PublicAgent, a multi-agent framework that addresses these limitations through targeted decomposition into specialized agents for intent clarification, dataset discovery, analysis, and reporting. This architecture maintains focused attention within agent contexts and enables validation at each pipeline stage. Benchmark evaluation across eight models and ablations across five representative models on 50 queries identify empirical design guidelines for multi-agent LLM systems in open-data analysis. The results show that specialization improves performance across the evaluated models, that discovery and analysis agents provide the most consistent value, and that report and intent agents show stronger model-dependent effects. Ablations further indicate that different agents mitigate distinct failure modes, with discovery and analysis failures often causing unusable outputs while report and intent removal more often degrades quality. These findings provide evidence for when and why specialization helps complex analytical workflows, while also highlighting the need for broader benchmarks, human validation, and continued evaluation as frontier models evolve.

1. Introduction

Open and public data serve as critical resources for improving transparency, supporting innovation, and promoting equitable development. When accessible to a broader audience, these datasets enable communities to tackle pressing issues, refine policies, and support informed discourse. Yet, the sheer volume, intricate structures, and dispersed nature of such data — exemplified by the U.S. Government's data.gov portal with nearly 300,000 datasets — impose heavy demands on users seeking to derive meaningful conclusions [1,2].

Realizing this potential requires overcoming substantial technical barriers. Users without data mining and analysis expertise encounter obstacles at every stage of the analytical workflow. Identifying pertinent datasets demands semantic search across scattered repositories with inconsistent metadata standards. Once located, datasets frequently arrive in heterogeneous formats that require normalization and schema mapping to resolve inconsistencies. Transforming raw data into insights involves statistical computation and code generation, skills inaccessible to journalists, policymakers, and community advocates without technical training. These compounding barriers restrict open data utilization

to a narrow technical audience, undermining the democratic potential of public information.

To address these accessibility challenges, large language models (LLMs) emerge as a promising solution with demonstrated capabilities in information retrieval, data manipulation, and code synthesis through natural language interfaces [3–8]. Prior efforts have applied LLMs to individual stages of data workflows: generating retrieval-augmenting knowledge, assessing dataset relevance, creating synthetic data, and evaluating code generation. However, these approaches address isolated components rather than the full analytical pipeline from ambiguous query to evidence-based report.

Applying LLMs to the complete analytical workflow reveals important limitations of single-model approaches. A complete workflow requires semantic search across repositories, schema mapping to diverse formats, statistical analysis with validation, and report synthesis with appropriate caveats—each demanding specialized reasoning patterns. When a single LLM handles all stages sequentially, attention dilutes across the growing context window, specialized skills interfere (semantic search reasoning differs from statistical analysis), and errors

* Correspondence to: University of North Texas, Discovery Park, E295A 3940 N Elm St Denton, TX, 76207, USA.

E-mail addresses: sina.montazeri@my.unt.edu (S. Montazeri), Yunhe.Feng@unt.edu (Y. Feng), Kewei.Sha@unt.edu (K. Sha).

propagate without detection. Consider a journalist asking “How prevalent is adult high blood pressure in the United States?” The system must clarify “adult” (age 18+?) and the operational meaning of “high blood pressure” in the selected dataset, search heterogeneous health repositories, map to correct schema fields, execute validated statistical code, and present results with dataset provenance and limitations. Asking one model to maintain coherent reasoning across these distinct competencies in a single long-context invocation can lead to semantic drift, factual errors, and incomplete analysis.

To address these limitations, we present PublicAgent, a multi-agent framework that decomposes the analytical pipeline into specialized agents with distinct responsibilities: intent clarification resolves query ambiguities, data discovery performs semantic search and metadata synthesis, data analysis generates and validates statistical code, and report generation synthesizes findings with appropriate caveats. This specialization is designed to reduce task interference, enable targeted validation at each stage, and maintain focused attention within agent contexts. Natural language interfaces at every stage reduce the need for SQL, Python, or API knowledge, improving access for users without technical expertise.

Our main contributions include:

- Five empirical design guidelines for multi-agent LLM systems in open-data analysis, covering: (1) value across the evaluated model strengths, (2) more consistent versus more model-dependent agent roles, (3) distinct failure mode mitigation, (4) task-complexity effects, and (5) model-aware architecture design.
- Design and implementation of PublicAgent, a multi-agent framework for open data analysis that supports systematic ablation studies across specialized agents for intent clarification, dataset discovery, analysis, and reporting.
- Empirical evaluation across five language models and 50 diverse queries, with ablations isolating each agent’s contribution and showing how specialization can address attention dilution, task interference, and error propagation in this benchmark setting.
- Characterization of barriers in end-to-end open data analysis — dataset discovery across heterogeneous repositories, schema mapping, and statistical validation — that motivate multi-agent decomposition while enabling broader access through natural language interfaces.

2. Related work

Natural language interfaces for data analysis represent a convergence of multiple research domains. Prior work has made substantial progress on individual components — semantic parsing, automated analysis, multi-agent coordination, query refinement, and report generation — but no integrated system addresses the complete pipeline from ambiguous natural language queries to comprehensive reports using discovered open datasets. We organize the related work into five key areas, identifying what each contributes and what gaps remain.

2.1. Natural language interfaces to databases

Early work on natural language interfaces to databases (NLIDB) focused on translating user queries into structured database queries. LUNAR demonstrated rule-based semantic parsing for database queries [9]. This foundation allowed for more sophisticated neural approaches in the subsequent years. Text-to-SQL systems like SQLNet [10] and RAT-SQL [11] use sequence-to-sequence models to map questions to SQL queries. These systems achieve substantial accuracy on benchmarks like Spider [12] and WikiSQL [13], although they typically assume a known database schema and focus on single-step query generation.

The integration of large language models (LLMs) has further advanced semantic parsing capabilities. BIRD [14] and DAIL-SQL [15]

leverage pre-trained models with few-shot prompting to handle complex queries across diverse schemas. These advances represent significant progress in query translation for known databases, yet remain limited to pre-existing schemas. Open data analysis requires additional capabilities: discovering relevant datasets across heterogeneous repositories and synthesizing metadata when publisher information is incomplete or inconsistent.

2.2. Automated data analysis and AutoML

Although semantic parsing assumes data access, automated analysis systems address the complementary challenge of extracting insights once datasets are available. Automated machine learning (AutoML) frameworks have evolved to address increasingly complex data science workflows. Early systems like Auto-WEKA [16], AutoSklearn [17] and TPOT [18] automate model selection and hyperparameter tuning for predictive tasks. More recent systems extend this automation to broader aspects of data analysis. AIDE [19] generates exploratory data analysis code, while Data Formulator [20] enables iterative chart specification through natural language. These systems excel at specific analytical tasks, but require users to provide datasets. They often focus on visualization or model training rather than comprehensive reporting.

Systems such as LIDA [21] and ChartLlama [22] move closer to end-to-end generation. LIDA generates visualizations from data through goal-driven exploration. ChartLlama supports chart understanding and generation using multimodal LLMs. Both systems demonstrate the potential of automated insight generation, yet they assume datasets are pre-loaded and focus on visualization rather than comprehensive analytical workflows spanning query clarification, dataset discovery, multi-faceted analysis, and narrative synthesis.

2.3. Multi-agent systems for complex tasks

Recognizing that isolated capabilities are insufficient for complex workflows, multi-agent architectures have emerged as effective strategies for decomposing tasks into specialized subtasks. This paradigm recognizes that diverse capabilities are better handled through specialization than monolithic approaches. MetaGPT [23] uses software development roles (architect, engineer, tester) to generate code collaboratively. AutoGen [24] provides a framework for conversational agents that coordinate through the transmission of messages. TaskWeaver [25] decomposes data analysis tasks into code snippets executed by specialized agents. These systems demonstrate performance improvements when tasks require multiple competencies.

The success of multi-agent collaboration extends to data science and information retrieval. Co-STORM [26] demonstrates multi-agent collaboration for information seeking. ChatDev [27] applies collaborative agents in software engineering. However, these systems primarily address code generation or web search within controlled environments where data sources, formats, and quality are known. Open data repositories present distinct challenges: semantic search across inconsistent metadata, format normalization without predefined schemas, and validation across varying data quality levels.

The orchestration of multi-agent systems varies in how control and coordination are implemented. HuggingGPT [28] positions an LLM as a central controller managing a four-stage pipeline — task planning, model selection, execution, and response generation — demonstrating sequential orchestration for multi-modal AI tasks. MegaAgent [29] extends this to large-scale coordination through dynamic task decomposition and parallel execution, minimizing human-designed prompts by having agents autonomously generate coordination procedures. AgentVerse [30] explores collaborative orchestration where expert agents are assembled into automatic systems exhibiting emergent behaviors in reasoning and tool utilization. Recent analyses [31–33] categorize

orchestration approaches along dimensions of centralization (centralized coordinators versus peer-to-peer), temporal structure (interleaved versus decomposition-first), and coordination protocols, finding that explicit orchestration mechanisms reduce error propagation and attention dilution in complex workflows.

Recent coding-agent and command-line frameworks, including Claude Code [34], Gemini CLI [35], and OpenClaw's ACP-based agent routing [36], provide tool access, file-system interaction, and workflow execution for software-development tasks. These systems improve reproducibility by exposing concrete execution traces and tool calls, but their primary unit of work is codebase modification or developer assistance. PublicAgent instead targets open-data analysis: it begins with ambiguous public-data questions, autonomously discovers candidate datasets, synthesizes metadata, executes statistical analyses, and produces traceable reports. This distinction motivates our domain-specific agent decomposition rather than adopting a general coding-agent framework as the evaluation target.

2.4. Query understanding and disambiguation

Even with multi-agent coordination and automated analysis capabilities, the full pipeline requires addressing ambiguities present in initial natural language queries. Effective data analysis begins with a precise query formulation. Ambiguities in initial queries can propagate through the analytical pipeline and compromise results. Research on query clarification addresses these issues through interactive dialogs. Elicitron [37] learns to ask clarifying questions for information retrieval tasks. ClariQ [38] provides a benchmark for conversational query refinement. These systems improve retrieval accuracy through iterative refinement. In database contexts, systems like NaLIR [39] identify ambiguous entities and relationships. They prompt users to resolve ambiguities before the execution of the query.

Large language models have enabled new approaches to query enhancement. Query2Doc [40] generates pseudo-documents to enhance retrieval. CoT-BERT [41] decomposes complex queries into reasoning chains. These approaches primarily target general search or retrieval tasks. Data analysis requires resolving domain-specific ambiguities — undefined age thresholds, imprecise medical terminology, and unclear temporal scopes — that directly affect dataset discovery and statistical computation accuracy.

2.5. Data-to-text and report generation

Finally, once data is discovered, analyzed, and validated, results must be synthesized into accessible reports for non-expert audiences. Automated report generation from structured data has progressed substantially from the early template-based approaches. Early data-to-text systems like FoG [42] and SumTime [43] relied on hand-made rules. The introduction of neural models such as LSTM encoders and Transformers enabled more flexible generation. Recent work on table-to-text generation includes TAPAS [44] for question answering over tables and GPT-3-based approaches for numerical reasoning [45]. These advances have improved fluency and coherence in generated text.

Applications in data analysis contexts have demonstrated practical value. Quill [46] generates narrative insights from business intelligence data. Narrativa [47] produces automated journalism from structured sources. These systems deliver compelling narratives, but assume clean, well-structured inputs and focus on linguistic fluency. Multi-stage analytical pipelines require maintaining traceability across components — query clarification, dataset provenance, analytical methods, validation results — while adapting technical content for non-expert audiences.

The preceding review reveals substantial progress on individual components—semantic parsing for known databases, automated analysis for pre-loaded datasets, multi-agent coordination for controlled environments, and report generation for clean inputs. However, no

system integrates these capabilities for heterogeneous open data repositories where datasets must be discovered across inconsistent metadata, schemas are unknown, and quality varies. Our framework addresses this gap through a multi-agent architecture designed specifically for open data challenges: intent clarification targeting analytical ambiguities, dataset discovery with metadata synthesis, validated statistical analysis, and traceable report generation. Through benchmark evaluation across eight models and systematic ablations across five representative models, we demonstrate when and why agent specialization provides value, characterizing the failure modes — attention dilution, task interference, error propagation — that necessitate decomposition for complex analytical workflows.

3. Methodology

Transforming natural language queries into evidence-based analytical reports requires bridging semantic ambiguity, heterogeneous data formats, statistical computation, and narrative synthesis. We decompose this task into specialized functions implemented by dedicated agents: intent clarification resolves query ambiguities, data discovery performs semantic search and metadata synthesis across repositories, data analysis generates and validates statistical code, and report generation synthesizes findings with appropriate limitations. This section formalizes the problem, describes each agent's implementation, and details the coordination mechanisms that maintain coherence across the pipeline.

3.1. Problem formulation

We formalize the task of converting a user's natural language query Q^u into a comprehensive report R derived from a collection of open and public datasets $D = \{D_1, \dots, D_n\}$. This process addresses the challenge of mapping unstructured queries to structured data across heterogeneous repositories. The overall solution is an orchestration function f_o that coordinates specialized agents:

$$R = f_o(Q^u, D) \text{ where } f_o \text{ coordinates } (f_q, f_d, f_x, f_g).$$

The orchestration function f_o manages specialized functions that handle query refinement, dataset discovery, data analysis, and report generation: f_q , f_d , f_x , and f_g , respectively.

These functions operate sequentially under orchestrator coordination to respect information dependencies: query refinement must precede dataset discovery to avoid propagating ambiguities, dataset discovery must inform analysis to ensure schema compatibility, and report generation requires all prior outputs for synthesis.

The query refinement function f_q improves the original query Q^u through clarification:

$$Q^e = f_q(Q^u).$$

The dataset discovery function f_d selects a relevant dataset $D_i \in D$ and enriches it with comprehensive metadata M synthesized from multiple sources: publisher-provided schemas, computed statistical summaries, structural information (column names, types, row counts), sample data excerpts, and dataset provenance details:

$$(D_i, M) = f_d(Q^e, D).$$

The data analysis function f_x takes as input the enhanced query, selected dataset, and its metadata:

$$E = f_x(Q^e, D_i, M),$$

where E represents a collection of analytical experiments, each implemented as an executable Python program that operates on D_i to extract insights aligned with Q^e . The function dynamically generates experiments based on query requirements—from simple aggregations to complex multi-step analyses involving filtering, grouping, and statistical computations.

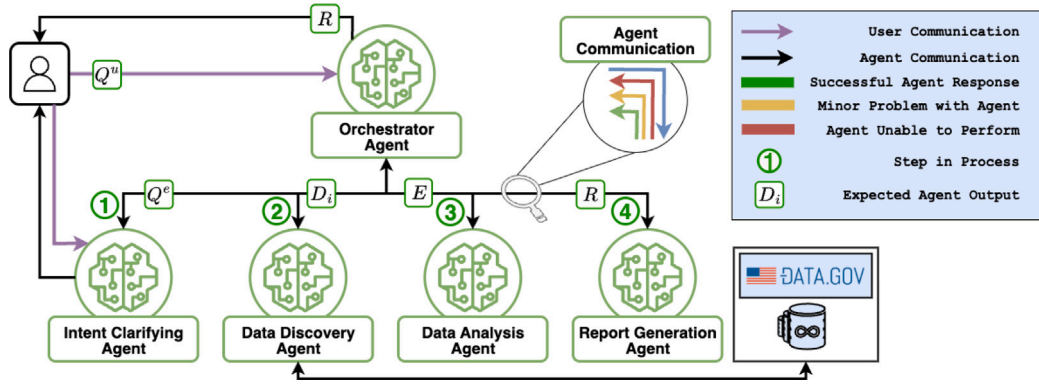


Fig. 1. Overall structure.

Finally, the report generation function f_g synthesizes these elements into the final report R :

$$R = f_g(Q^u, Q^e, D_i, E).$$

To evaluate the report, we define a quality function $Q(R)$ assessing its performance across multiple dimensions:

$$Q(R) = \frac{1}{4}(F(R) + C(R) + V(R) + H(R)),$$

where $F(R)$, $C(R)$, $V(R)$, and $H(R)$ are scores (1–10) for factual consistency, completeness, relevance, and coherence, respectively, derived from rubric-based judgments. Each component of the pipeline thus contributes to producing a report R that maximizes $Q(R)$ for a given query q .

Each component thus contributes to a coherent workflow from query to report.

3.2. Framework overview

Building on the problem formulation, PublicAgent implements f_o through a multi-agent pipeline that distributes responsibilities across specialized agents, each optimized for distinct aspects of the task. This decomposition exploits domain-specific processing strengths over monolithic agents, which falter when handling varied competencies such as linguistic refinement, data retrieval, statistical computation, and narrative synthesis.

PublicAgent proceeds sequentially to respect information dependencies: query clarification must precede dataset discovery to avoid propagating ambiguities; dataset discovery must inform analysis to ensure schema compatibility; and report generation requires all prior outputs for synthesis. Semantic consistency is maintained via structured message passing and summarization, where each agent validates its inputs and outputs against workflow invariants such as query intent and dataset provenance.

The implementation uses a custom orchestration layer built on LangGraph ReAct-style agents and LangChain tool interfaces [48]. Each specialized agent is instantiated as an independent tool-using agent with a domain-specific prompt, task-management state, short-term summarization, and access only to the tools required for its stage. The orchestrator invokes these agents sequentially and passes structured artifacts between stages rather than relying on a single shared conversation. This design differs from general coding-agent interfaces because the workflow state is organized around open-data analysis artifacts: clarified query, selected dataset, synthesized metadata, executable experiments, and final report.

Fig. 1 illustrates this structure, with agents for intent clarification (A_{IC}), data discovery (A_{DD}), data analysis (A_{DA}), and report generation (A_{RG}) operating under an orchestrator. This setup preserves fidelity from Q^u to R , aligning with the functions defined in the problem formulation.

3.3. Orchestrator agent

The orchestrator implements the coordination for f_o through sequential invocation of specialized agents, ensuring diverse capabilities required for the workflow are applied while preserving coherence. It organizes the process into stages aligned with f_q , f_d , f_x , and f_g and validates progress at each stage through evaluations of outputs such as refined queries, selected datasets, analysis results and generated reports.

The orchestrator passes context between stages and maintains information like original and enhanced queries, dataset details, and analysis outcomes. In the event of inconsistencies or errors, it identifies issues and initiates retries or adjustments, categorizing problems to apply appropriate resolutions: temporary failures lead to re-attempts, while persistent mismatches prompt re-evaluation of prior stages.

This approach allows reliable progression from Q^u to R , integrating the formalized functions into an operational sequence that adapts to potential disruptions in the multi-agent interaction.

3.4. Intent clarifying agent

Natural language queries for data analysis often contain colloquial terms, ambiguous scopes, and implicit assumptions that prevent effective dataset retrieval and analysis, as terms like “adults”, “high blood pressure”, or “recent” lack the precision necessary for matching dataset schemas or formulating analytical operations. Prior work either ignores this ambiguity (leading to semantic mismatches) or requires users to learn structured query languages (creating accessibility barriers), so the challenge is automatically transforming informal language into scientifically precise specifications while preserving user intent and conversational accessibility.

The intent clarifying agent implements f_q by identifying and resolving ambiguities in Q^u to produce an enhanced query Q^e . The agent detects up to three critical ambiguities — imprecise terms, unclear temporal or geographic scopes, and undefined thresholds — and proposes precise substitutions framed as confirmation questions. This iterative process continues until all identified issues are resolved.

Consider a user query Q^u : “How prevalent is high blood pressure among adults in the US?” The agent detects ambiguities in “adults” and “high blood pressure”. In this running example, we use a single operational definition throughout the pipeline: adults are individuals aged 18 years and older, and high blood pressure is represented by the selected dataset’s binary diagnosed-hypertension indicator, $x_{\text{hypertension}} = 1$. If a different dataset instead provides measured systolic and diastolic readings, the analysis must explicitly apply the dataset’s documented threshold rather than treating a diagnosis field and a measurement threshold as interchangeable. After confirmation, the enhanced query Q^e becomes:

Q^e = “What proportion of individuals aged 18 and older have diagnosed hypertension in the US?”

Table 1Components of standard metadata M generated by the data discovery agent.

Component	Description
First 5 rows	Initial rows offering a snapshot of dataset content.
Column names	List of all column headers in the dataset.
Number of rows	Total count of data entries.
Head output	Summary of the first five rows, including values for all columns.
Tail output	Summary of the last five rows, including values for all columns.
Statistical description	Descriptive statistics (e.g., mean, median, standard deviation) for numerical columns.
Unique column values	Up to ten unique values per column to show data diversity.
Publisher metadata	Publisher-provided codebook with column descriptions and data types.

Here, “adults” maps to individuals aged 18 and older, and “high blood pressure” maps to the diagnosed-hypertension indicator $x_{\text{hypertension}} = 1$.

This approach bridges colloquial language and analytical precision without requiring domain expertise, ensuring Q^e retains the intent of Q^u while providing a stable operational definition for dataset discovery and analysis.

3.5. Data discovery agent

Open data repositories expose datasets through heterogeneous APIs with inconsistent metadata standards, varying response formats, and unpredictable quality, requiring precise queries like “diagnosed hypertension prevalence in individuals aged 18+” to be mapped to relevant datasets across repositories with different terminologies, granularities, and access patterns. Existing approaches either rely on manual dataset curation (not scalable) or simple keyword matching (semantically inadequate), so the challenge is automated semantic search that handles vocabulary mismatches, evaluates dataset relevance, adapts to format inconsistencies, and synthesizes comprehensive metadata from fragmented sources.

The data discovery agent implements f_d to retrieve and prepare a relevant dataset D_i with enriched metadata M . It generates search terms from Q^e and queries repositories such as data.gov, evaluating results for semantic alignment with query intent. When multiple candidates exist, the agent selects the most suitable by domain relevance and schema compatibility; when none match, it adaptively broadens search parameters (e.g., expanding geographic scope from county to state level). The agent prioritizes CSV formats for compatibility and employs fallback mechanisms to extract direct download links when API responses are non-standard.

Metadata synthesis integrates publisher-provided information with insights from dataset inspection, and Table 1 outlines the standard components of metadata M generated by the agent.

Error handling distinguishes recoverable issues (transient download failures) from fundamental mismatches (semantic incompatibility with Q^e), ensuring reliable alignment with the query and preparing the dataset for subsequent analysis.

3.6. Data analysis agent

Translating a natural language query into executable analytical code requires bridging semantic intent with computational operations, complicated by heterogeneous dataset schemas, diverse analytical requirements, and validation needs. A query like “diagnosed hypertension prevalence by age subgroups” must be decomposed into specific computations (filtering, aggregation, subgroup analysis), mapped to correct dataset columns, implemented as executable Python code, and validated for correctness. Existing solutions either require manual

Table 2

Sample experiment outputs for hypertension prevalence query.

Experiment	Output
Adult diagnosed-hypertension prevalence	$P_{\text{hypertension}} = 0.32$ (32% of adults)
Prevalence, ages 18–44	$P_{\text{subgroup}} = 0.15$ (15% of subgroup)
Prevalence, ages 45–64	$P_{\text{subgroup}} = 0.38$ (38% of subgroup)
Prevalence, ages 65+	$P_{\text{subgroup}} = 0.54$ (54% of subgroup)

coding (inaccessible to non-experts) or apply rigid templates (inflexible for diverse queries), so the challenge is automated generation of multi-step analytical workflows that adapt to varied schemas, validate intermediate outputs, and handle data quality issues.

The data analysis agent implements f_x to transform the enhanced query Q^e , dataset D_i , and metadata M into verified insights through a structured analytical pipeline. The agent inspects M (column names, types, statistical summaries) to map query requirements to dataset structure, then decomposes Q^e into discrete experiments representing specific computations.

Consider the enhanced query “What proportion of individuals aged 18 and over have diagnosed hypertension in the US?” The agent first defines the adult subset

$$D_i^{18+} = \{x \in D_i \mid x_{\text{age}} \geq 18\}.$$

It then formulates experiments such as:

- Compute adult diagnosed-hypertension prevalence:

$$P_{\text{hypertension}} = \frac{|\{x \in D_i^{18+} \mid x_{\text{hypertension}} = 1\}|}{|D_i^{18+}|}$$

- Calculate prevalence by adult age subgroups (e.g., 18–44, 45–64, 65+):

$$P_{\text{subgroup}}(S) = \frac{|\{x \in D_i \mid x_{\text{hypertension}} = 1 \wedge x_{\text{age}} \in S\}|}{|\{x \in D_i \mid x_{\text{age}} \in S\}|},$$

$$S \in \{[18, 44], [45, 64], [65, \infty)\}.$$

The agent translates these mathematical formulations into executable code operating on the dataset. Fig. 2 demonstrates how the overall prevalence computation is implemented:

These experiments are executed as self-contained Python programs on a preloaded dataframe with pandas preimported, ensuring no modifications or variables persist between runs to prevent unintended side effects.

The results of each experiment are captured for the specified variables and evaluated for reasonableness. Anomalies, such as zero values indicating potential filtering errors, trigger diagnostic checks. If parsing issues arise (e.g., mixed separators or encoding errors), the agent adapts by testing alternative configurations to ensure reliable handling of heterogeneous data formats. Table 2 illustrates the sample results of these experiments.

The process iterates until all experiments are completed and validated, aggregating findings traceable to their hypotheses, producing results semantically aligned with Q^e , statistically reliable, and prepared for report generation.

3.7. Report generation agent

Synthesizing analytical results into accessible reports requires balancing technical precision with readability for diverse audiences, complicated by integrating information from multiple pipeline stages. A report must incorporate the original query context, clarified specifications, dataset provenance, analytical methods, results, limitations, and conclusions while adapting language complexity for non-expert readers without sacrificing accuracy. Existing systems either produce technical outputs inaccessible to general audiences or oversimplified summaries

```
# compute overall hypertension prevalence
adults = df[df['age'] >= 18]
hypertension_cases = adults[adults['hypertension'] == 1]
overall_prevalence = len(hypertension_cases) / len(adults)
```

Fig. 2. Example Python code generated by the data analysis agent for computing hypertension prevalence.

Table 3

Sections of the report generated by the report generation agent.

Section	Description
Title	A concise heading summarizing the analysis focus.
Summary for Non-Experts	A 1-2 paragraph overview explaining main findings in simple language, assuming high school-level knowledge.
Analysis Assumptions	Main assumptions underlying the analytical approach, detailing conditions presumed during processing.
Analysis Definitions	Explanations of technical terms and concepts used in the analysis, adapted for accessibility.
Experiments	Detailed account of conducted experiments, including Experiment Description, Narrative of each experiment's purpose and methodology. Columns Used, Specific dataset columns utilized in each experiment.
Limitations	A 1-2 paragraph discussion of potential uncertainties, biases, or constraints in the analysis.
Conclusions	A 1-2 paragraph synthesis of main insights derived from the experiments.
Dataset Link	Reference to source dataset for traceability and reproducibility.

that lack rigor, so the challenge is automated report generation that maintains traceability across workflow stages, adapts terminology to audience expertise, and structures information for coherent narrative flow.

The report generation agent implements f_g to synthesize inputs from previous stages into a structured report R balancing accessibility and precision. It integrates the original query Q^u , enhanced query Q^e , dataset information, and analysis results into distinct sections outlined in Table 3.

The agent employs adaptive language modeling to simplify technical terminology for non-expert audiences while preserving analytical precision. It validates that each section is substantive, logically connected, and consistent with prior workflow outputs to prevent information loss during synthesis. Maintaining explicit traceability from Q^u through Q^e , dataset selection, analytical experiments, to final conclusions enables verification and reproducibility while remaining accessible to users without technical expertise.

3.8. Agent tools

While large language models possess strong general reasoning capabilities, multi-agent systems require specialized tools to address coordination challenges emerging from task decomposition. When a complex analytical workflow is divided among specialized agents, three fundamental problems arise: (1) agents must maintain coherent state across sequential steps without losing critical information, (2) agents must validate intermediate outputs to prevent error propagation, and (3) agents must interface with external systems (datasets, execution environments) that standard language models cannot directly access.

PublicAgent provides four tool categories — task management, reasoning augmentation, execution isolation, and data integration — purpose-built for the coordination problems arising when decomposing data analysis into specialized agent responsibilities. Each tool category supports agents differently:

- **Task Management:** All agents use task management to coordinate workflows. The intent clarifying agent tracks query refinement steps, the data discovery agent directs dataset search and retrieval, the data analysis agent orchestrates experiment planning and validation, and the report generation agent tracks section creation.
- **Reasoning and Validation:** Each agent applies reasoning tools to its domain. The intent clarifying agent resolves query ambiguities, the data discovery agent validates dataset relevance, the data analysis agent checks experiment outcomes, and the report generation agent confirms report coherence.
- **Execution Environment:** Only the data analysis agent requires code execution, using an isolated environment to run analytical experiments on datasets.
- **Data Integration:** Three agents interact with external data. The data discovery agent queries repositories and synthesizes metadata, the data analysis agent processes diverse dataset formats, and the report generation agent incorporates dataset details into the final output.

3.8.1. Task management system

Language models excel at generating responses to prompts but lack native state management for multi-step processes. When an analysis requires multiple dependent operations — metadata inspection, experiment formulation, execution, validation — agents need explicit mechanisms to track progress, manage dependencies, and coordinate retries. Without structured task management, agents either attempt all steps in a single invocation (overwhelming context and causing errors) or lose track of remaining work.

The task management system maintains a dynamic task list with unique identifiers, status tracking (pending, in-progress, completed), and dependency enforcement. Agents query the system to determine remaining work, mark tasks complete when finished, and spawn new tasks when dependencies are satisfied. This transforms stateless language model invocations into stateful workflow execution. Externalizing state management allows agents to focus on executing individual tasks rather than tracking the entire workflow. Dependency enforcement prevents agents from attempting operations before prerequisites are met (e.g., running analysis before dataset retrieval), while error recovery through task re-queueing enables graceful handling of transient failures without manual intervention. The system scales to complex queries by decomposing them into manageable subtasks: queries requiring multiple dataset formats, geographic aggregations, or temporal comparisons routinely generate 15–20 tasks with intricate dependency chains, which the system executes reliably through incremental progression and isolated failure recovery.

Consider the enhanced query “What proportion of individuals aged 18 and over have diagnosed hypertension in the United States?” The system generates and tracks tasks for the data analysis stage as shown in Table 4.

Each task has a unique identifier and status tracking. If validation fails, the system re-queues the task with adjusted parameters to ensure reliable progression.

3.8.2. Thinking and quality check tools

Multi-agent workflows accumulate context as information flows between agents—original query, refined query, dataset metadata, experiment results. This growing context window causes attention dilution in

Table 4

Example task management for hypertension analysis query.

ID	Status	Task description
1	Completed	Analyze dataset metadata (columns, types, statistics)
2	Pending	Formulate experiment for adult hypertension prevalence
3	Pending	Formulate experiment for adult age-subgroup prevalence (18–44, 45–64, 65+)
4	Pending	Execute adult prevalence computation, validate for anomalies
5	Pending	Execute subgroup prevalence, validate for reasonableness
6	Pending	Diagnose parsing issues if format inconsistencies occur

the transformer attention mechanism. At each position in the sequence, the model computes how much to “attend to” every other position when processing information. The attention weight determines what fraction of each context position contributes to the current computation. For a query position i attending to key position j in a sequence of length N , where q_i and k_j are learned representations and d is the embedding dimension, the attention weight is computed as:

$$\alpha_{ij} = \frac{\exp(q_i^\top k_j / \sqrt{d})}{\sum_{\ell=1}^N \exp(q_i^\top k_\ell / \sqrt{d})}.$$

As N grows, the normalizing denominator accumulates more terms, forcing each individual attention weight α_{ij} to become smaller since all weights must sum to 1. This spreads attention across more positions, diluting focus on relevant information [49,50]. Additionally, intermediate outputs such as computed statistics may contain errors that propagate if not detected early.

The thinking tool prompts agents to explicitly articulate their current observation, planned action, and reasoning before proceeding, forcing distillation of relevant context. The quality check tool complements this by applying domain-specific validation rules to intermediate outputs, flagging zero-valued aggregations suggesting data filtering errors and checking for statistical plausibility. Explicit thinking breaks the reasoning process into observable steps, making errors detectable and reducing reliance on implicit context tracking. Quality checks catch errors before they propagate to downstream agents, preventing cascading failures. Together, these tools compensate for the attention dilution and error accumulation inherent in long-context, multi-step workflows.

3.8.3. Analytical execution environment

Language models cannot directly execute code or access computational environments, yet data analysis requires running Python scripts on actual datasets. The execution environment provides the data analysis agent with isolated execution capability. It maintains a preloaded dataframe with necessary libraries while enforcing strict constraints to prevent persistent changes or unauthorized imports. Each experiment thus runs independently, avoiding unintended interactions between sequential analyses. The environment captures results for specified variables and applies validation checks to confirm computational accuracy. Providing controlled code execution with safety guarantees transforms language model suggestions into verified analytical results.

3.8.4. Online open data integration

Open data repositories expose datasets through heterogeneous APIs with varying formats, metadata standards, and access patterns. Language models cannot directly query external services or parse diverse response formats. The data integration component manages dataset queries to repositories such as data.gov, validating format consistency, and synthesizing metadata from multiple sources. It caches previously retrieved datasets to improve performance and adapts to format variations by extracting direct download links from non-standard responses. This combination delivers datasets in consistent formats ready for analysis, abstracting away the complexity of heterogeneous data sources.

Table 5

Analytical intricacy stratification with examples.

Intricacy level	Example query
Easy (Direct Aggregation)	“Which state has the highest rate of obese adults?”
Medium (Filtering and Grouping)	“How many electric cars in Seattle are fully electric, not hybrids?”
Hard (Trend Analysis)	“Has the obesity rate in California gone up or down over the years?”

Table 6

Benchmark query domains and representative examples.

Domain	Example query
Health Epidemiology	“How prevalent is adult high blood pressure in the United States?”
Environmental Monitoring	“Which NYC neighborhood has the worst nitrogen dioxide air pollution?”
COVID-19 Analysis	“How do COVID-19 death rates compare between unvaccinated and vaccinated people in Chicago in 2022?”
Transportation	“Which area in Washington has the most electric cars?”
Campaign Finance	“How have total campaign contributions changed from 2020 to 2025 in Washington state?”
Public Health	“Which racial or ethnic group has had the highest cancer death rate in NYC since 2007?”

4. Evaluation

We evaluate PublicAgent on two axes: report quality for real public-data questions and ablations testing whether specialized agents are needed compared to a single-orchestrator baseline. This setup measures both performance on open-data queries and the contribution of each component.

4.1. Benchmark design: Queries, coverage, and justification

The benchmark includes $|Q| = 50$ queries reflecting real public-data use cases. Each query represents how non-experts interact with open data: clarifying intent, finding datasets, analyzing tables, and presenting results with sources and caveats. Queries span domains and complexity levels instead of focusing on a single task such as text-to-SQL. They cover health, environment, transportation, campaign finance, public health, and COVID-19 (Table 6), matching areas common across open data portals (e.g., Data.gov, NYC Open Data). Each query links to at least one dataset with a clear schema and documented provenance (see Table 5).

We selected datasets using portal metadata such as topic tags, geography, and time coverage from sources like Data.gov. Candidate datasets passed filters for tabular format, adequate coverage, and open licenses. Existing benchmarks for text-to-SQL (Spider [12], WikiSQL [13], BIRD [51]) explicitly avoid ambiguous queries — Spider annotators were instructed to exclude questions that are “vague or too ambiguous” [12] — and provide known database schemas, resulting in scientifically formulated questions with 40%–50% column-name overlap. While benchmarks like KaggleDBQA [52] introduced more natural phrasing (6.8% column mention), they still assume pre-identified databases. No existing benchmark evaluates the full pipeline from colloquial ambiguous queries through autonomous dataset discovery to validated analysis and reporting. This ensures each question can be answered end-to-end with verifiable public data, supporting reproducibility and real-world relevance.

Benchmark design determines what abilities we measure. Prior datasets like Spider and BIRD focus on semantic parsing accuracy, testing text-to-SQL skill but not dataset discovery or reporting [12,51]. Our benchmark includes the full pipeline from ambiguous natural language to evidence-backed reports. The stratified complexity captures

Table 7

Language model configuration for evaluation. Entries marked with † indicate that exact parameter counts are not publicly disclosed by the provider.

Model	Parameters	Temperature
Google Gemini 2.5 Pro	Large†	1.0
Google Gemini 2.5 Flash	Large†	1.0
Google Gemini 3 Flash Preview	Large†	1.0
Meta Llama 3.3 70B Instruct	70B	0.9
MiniMax M2.7	Large†	1.0
OpenAI GPT-4o Mini	Small†	1.0
OpenAI GPT OSS 120B	120B	0.5
X.AI Grok 3 Mini	Small†	1.0

reasoning differences among aggregation, filtering, and temporal analysis [53]. Domain diversity tests generalization, since models trained on narrow distributions often fail on unfamiliar topics.

4.2. Models and decoding temperatures

We evaluate eight models from multiple sources: OpenAI GPT OSS 120B, Meta Llama 3.3 70B Instruct, OpenAI GPT-4o Mini, X.AI Grok 3 Mini, Google Gemini 2.5 Pro, Google Gemini 2.5 Flash, Google Gemini 3 Flash Preview, and MiniMax M2.7 (Table 7). This expanded set includes both the original five representative models used for the ablation study and three newer models added in the revision. Differences in training, provider, and scale reduce dependence on one model family, but these models do not exhaust the space of current or future systems. We therefore interpret the results as evidence from a diverse benchmark model set rather than as proof that the same effects hold universally. Each model's decoding temperature τ is tuned on a held-out set to balance quality and consistency.

Temperature affects the balance between determinism and diversity [54]. Low τ favors consistent but repetitive text. High τ increases variation but can cause factual errors. We select τ to reduce output variance while keeping analytical completeness. Models like Llama, tuned for instruction following, often need higher τ to avoid rigid patterns, while RLHF-trained models may perform best at lower τ [55,56]. The final values in Table 7 reflect each model's optimum setting.

4.3. Metrics and mathematical formulation

Each system produces a report R for query q . For benchmark-quality scoring, two independent LLM judges (GPT 5 & GPT 5.4 mini) score factual consistency F , completeness C , relevance V , and coherence H on a 1–10 scale using explicit rubrics, and we average their criterion scores. For ablation comparisons, an independent LLM judge evaluates blinded report pairs using the same criteria.

- **Factual Consistency (1–10):** Does the report maintain internal logic without contradictions? Do the data points match the analysis? Are claims supported by the evidence presented?
- **Completeness (1–10):** Does the report cover key aspects of a data analysis? Are all necessary components present for a thorough analysis?
- **Relevance (1–10):** Does the report directly answer the question? Is the content focused and on-topic?
- **Coherence (1–10):** Does the report have logical flow and readability? Is it well-structured and easy to follow?

The overall score is computed as the average:

$$Q(R) = \frac{1}{4}(F(R) + C(R) + V(R) + H(R)).$$

For ablations, we compare full outputs R^{full} with ablated versions R^{abl} using blinded pairwise judgments. For each query q and quality criterion k , the judge assigns a score $s_k(q)$ to the full system: 1 if R^{full}

wins, 0 if R^{abl} wins, and 0.5 for a tie. The tie-inclusive win rate is therefore

$$W_k = \frac{1}{|Q|} \sum_{q \in Q} s_k(q), \quad s_k(q) \in \{0, 0.5, 1\}.$$

We report W_k across all 50 benchmark queries, with ties counted as half wins.

4.4. LLM-as-judge protocol and related practice

LLM judges score reports and decide A/B preferences using a fixed rubric. This approach aligns with MT-Bench and Chatbot Arena, which show strong correlation between LLM and human ratings [57]. We apply position randomization, length normalization, and explicit rubrics following AlpacaEval 2.0 and Prometheus [58,59]. These steps reduce bias and improve reproducibility [60,61].

LLM judges are efficient for multi-dimensional quality evaluation. Human review of long-form text is costly and inconsistent. LLMs apply criteria uniformly and provide explanations for their scores [57]. Our rubrics define concrete evidence — citations, numerical checks, logical order — rather than vague descriptors, improving reliability [62]. While not perfect human substitutes, LLM judges perform well on structural criteria like coherence and completeness.

For pairwise ablation comparisons, the judge evaluates both reports on four criteria with explicit instructions:

- **Factual Consistency:** Which report maintains better internal logic without contradictions? Do the data points match the analysis better?
- **Completeness:** Which report better covers intent clarification, data sourcing, analysis steps, and conclusions?
- **Relevance:** Which report more directly answers the question without unnecessary fluff?
- **Coherence:** Which report has better logical flow and readability?

These qualitative comparisons are converted to quantitative metrics through the tie-inclusive scoring rule defined above. Aggregating $s_k(q)$ across all queries yields the win rate for each evaluation dimension.

Each judge responds with structured JSON output indicating the winner for each criterion with explanation, plus an overall winner determination. This structured format ensures consistent evaluation across all comparisons and enables quantitative aggregation of win rates.

4.5. Ablations: Testing the multi-agent hypothesis

We remove one agent at a time — intent clarification, data discovery, analysis, or reporting — while keeping the orchestrator to fill the gap. This isolates each agent's contribution.

Ablation testing isolates component effects in complex systems [63]. It attributes performance changes to architecture, not confounding factors such as model size. We maintain comparable workflows, vary one component per test, and measure effect size through scores and pairwise preferences. Consistent effects across model families support general conclusions.

Across the five original models used for ablation, the full system wins most pairwise comparisons: analysis-agent ablations lose 76%–100% and discovery-agent ablations lose 79.5%–98%. These results provide evidence that specialized agents improve consistency across the evaluated architectures and sizes; broader claims require additional models and larger query sets.

4.6. Comparative protocol and validity

For each $q \in Q$, we generate both full and ablated reports. Judges assign rubric scores and blinded preferences to compute $Q(R)$ and W_k . We reduce bias by randomizing order, allowing ties, and anchoring rubrics. Some residual bias may persist because the evaluation still depends on an LLM judge rather than a fully independent human panel. The 50-query set is balanced and grounded in open-data portals, but its size limits statistical power and generalizability beyond the six domains studied. Expanding the benchmark, adding human evaluation on a representative subset, and reporting uncertainty intervals would strengthen future validation. Within these limits, consistent ablation outcomes across the evaluated models suggest that agent specialization improves open-data analysis and reporting beyond what scale alone achieves.

5. Results

Our findings confirm that PublicAgent performs strongly across all evaluated models and that its multi-agent structure offers measurable advantages.

5.1. Benchmark performance analysis

Table 8 shows the performance of all models across factual, completeness, relevance, and coherence criteria.

OpenAI GPT OSS 120B ranks highest with an overall score of 8.2, showing strong coherence (9.2) and relevance (8.5). This indicates consistent and well-structured analytical outputs. Google Gemini 2.5 Pro (7.2 overall) and MiniMax M2.7 (7.1 overall) follow closely, with MiniMax showing particularly strong completeness (7.6).

OpenAI GPT-4o Mini performs competitively at 6.8, despite being smaller in scale. Its strong relevance (7.9) and coherence (8.1) scores show that compact models can still excel in structured reasoning tasks when properly supported by agent architecture. The newly added Gemini 3 Flash Preview reaches 6.3 overall, while Gemini 2.5 Flash scores 5.5 overall, indicating that newer model releases do not automatically dominate older or larger systems in tool-using open-data workflows.

Meta Llama 3.3 70B performs poorly (4.7), demonstrating that parameter count alone does not predict success in multi-agent analytical settings. Across both original and newly added models, the results suggest that tool-use reliability, dataset-discovery behavior, and fit with the workflow matter alongside base model capability.

Completeness remains a difficult criterion, with wide score variation (3.3–7.9). This pattern shows that generating thorough reports across multiple data dimensions is one of the most demanding aspects of the task.

5.2. Performance across question difficulty

Table 9 presents results by question difficulty that we manually assigned during the benchmark dataset creation process, based on perceived complexity of the data analysis task.

OpenAI GPT OSS 120B maintains stable results across all difficulty levels (8.1–8.2), indicating consistent reasoning performance. MiniMax M2.7 also remains stable from medium to hard questions (6.8 in both categories) and performs especially well on easy questions (7.7). Gemini 2.5 Pro and GPT-4o Mini both show mild declines on hard questions, though they handle easier queries well.

The added Gemini Flash models show different difficulty profiles: Gemini 3 Flash Preview is relatively balanced across easy and medium questions, while Gemini 2.5 Flash performs better on medium and hard questions than on the easy subset because several easy questions triggered dataset-discovery or workflow failures. Meta Llama 3.3 70B and X.AI Grok 3 Mini show modest variation across difficulty levels. Overall, difficulty alone does not fully explain performance variation; query structure, dataset availability, and model-specific tool-use behavior also play important roles.

Table 8

Benchmark performance results by model and quality criteria.

Model	Factual	Complete	Relevant	Coherent	Overall
Google Gemini 2.5 Pro	6.8	6.3	7.4	8.2	7.2
Google Gemini 2.5 Flash	5.0	5.9	4.9	6.4	5.5
Google Gemini 3 Flash Preview	6.1	6.8	5.8	6.6	6.3
Meta Llama 3.3 70B Instruct	4.7	3.3	4.1	6.7	4.7
MiniMax M2.7	6.8	7.6	6.4	7.3	7.1
OpenAI GPT-4o Mini	5.5	5.6	7.9	8.1	6.8
OpenAI GPT OSS 120B	7.1	7.9	8.5	9.2	8.2
X.AI Grok 3 Mini	5.1	4.8	5.2	7.8	5.8

Table 9

Benchmark performance by question difficulty.

Model	Easy	Medium	Hard	Overall
Google Gemini 2.5 Pro	7.3	7.4	6.5	7.2
Google Gemini 2.5 Flash	4.1	6.4	5.6	5.5
Google Gemini 3 Flash Preview	6.6	6.5	5.6	6.3
Meta Llama 3.3 70B Instruct	4.5	5.3	3.9	4.7
MiniMax M2.7	7.7	6.8	6.8	7.1
OpenAI GPT-4o Mini	7.1	7.1	5.8	6.8
OpenAI GPT OSS 120B	8.1	8.2	8.2	8.2
X.AI Grok 3 Mini	5.2	6.4	5.1	5.8

5.3. Criterion-specific ablation effects

Table 10 summarizes how removing agents affects quality dimensions. Each agent influences distinct aspects of performance, showing complementary rather than redundant roles.

Win rates indicate the percentage of queries where the full multi-agent system outperforms the ablated version missing one agent; ties count as 0.5. The dashes for the Grok No Report row indicate that the corresponding ablation run did not produce complete judgeable outputs, so criterion-level win rates are not reported for that condition.

Removing the analysis agent reduces factual accuracy most sharply, with win rates from 42% (GPT-4o Mini) to 96% (GPT OSS 120B). This wide variation shows that models differ in how they rely on structured reasoning—some need agents for factual verification, others for structural organization. The analysis agent consistently supports completeness in this benchmark (88%–100% win rates), indicating its importance for full analytical coverage across the evaluated models.

The discovery agent achieves near-perfect completeness gains (96%–100% win rates) and strong relevance effects (61%–100% win rates). This pattern shows that proper dataset discovery improves both scope and precision. The dual effect confirms that discovery provides the right information and keeps analysis on topic.

Report generation has balanced effects across the models with complete report-ablation data, with win rates from 44%–96% (factual), 63%–100% (completeness), 62%–100% (relevance), and 46%–100% (coherence). This uniformity supports the agent's integrative role. The large performance gap between models (GPT OSS 120B: 96%–100% vs GPT-4o Mini: 44%–68%) suggests that smaller models may depend more on this agent for coherent synthesis.

Intent clarification strongly improves completeness (73%–100% win rates) and shows moderate effects elsewhere (61%–98%). This pattern supports the idea that clear query definition benefits all downstream processing without targeting any single quality dimension.

5.4. Model-specific agent dependencies

Agent dependencies differ sharply across models, showing that optimal architecture depends on base model characteristics.

OpenAI GPT OSS 120B benefits uniformly from all agents (92%–100% win rates), showing broad synergy without overreliance on any one component. No single agent is disproportionately important. Even this strong base model (8.2/10) cannot compensate for missing agents through its own capabilities.

Table 10
Full system win rates by criterion, model, and ablation type.

Model	Ablation	Factual	Complete	Relevant	Coherent
Google Gemini 2.5 Pro	No Analysis	71.4%	79.6%	69.4%	71.4%
	No Discovery	69.4%	83.7%	83.7%	79.6%
	No Report	53.1%	63.3%	69.4%	55.1%
	No Intent	61.2%	73.5%	69.4%	73.5%
Meta Llama 3.3 70B Instruct	No Analysis	76.7%	100.0%	79.1%	97.7%
	No Discovery	69.2%	82.1%	61.5%	87.2%
	No Report	57.8%	93.3%	73.3%	93.3%
	No Intent	67.3%	89.8%	79.6%	91.8%
OpenAI GPT-4o Mini	No Analysis	42.0%	88.0%	76.0%	76.0%
	No Discovery	64.0%	96.0%	92.0%	88.0%
	No Report	44.0%	68.0%	62.0%	46.0%
	No Intent	63.3%	73.5%	63.3%	69.4%
OpenAI GPT OSS 120B	No Analysis	96.0%	96.0%	80.0%	92.0%
	No Discovery	94.0%	100.0%	100.0%	100.0%
	No Report	96.0%	100.0%	100.0%	100.0%
	No Intent	98.0%	98.0%	98.0%	98.0%
X.AI Grok 3 Mini	No Analysis	82.0%	100.0%	98.0%	100.0%
	No Discovery	88.0%	100.0%	96.0%	98.0%
	No Report	–	–	–	–
	No Intent	90.0%	100.0%	96.0%	98.0%

In contrast, Meta Llama 3.3 70B shows selective dependence on analysis and report agents, with 76%–100% win rates when analysis is removed and 57%–93% for report generation. The particularly strong reliance on completeness (100% analysis, 93% report) shows the model struggles with comprehensive output generation and needs substantial architectural support.

OpenAI GPT-4o Mini depends most on the analysis agent for factual grounding, showing only 42% wins on factual consistency when this agent is removed—the lowest across the ablation models. This reveals a specific weakness in maintaining factual accuracy during complex analytical tasks. The analysis agent provides critical verification that the base model cannot perform independently, showing that smaller models can achieve competitive overall performance through strategic agent support that compensates for specific weaknesses.

X.AI Grok 3 Mini performs well in the completed ablations (82%–100% win rates) and achieves perfect completeness scores (100%) when the analysis agent is removed. However, the report-generation ablation did not yield complete judgeable outputs, so we do not draw criterion-level conclusions about Grok’s dependence on the report agent.

5.5. Design principles for agentic AI systems

Our results suggest five empirical design guidelines for multi-agent architectures in open-data analysis. These guidelines are primarily supported by the five-model ablation study, with the expanded eight-model benchmark providing additional context about model variation.

5.5.1. Specialization provides value independent of model strength

Within the five-model ablation set, multi-agent architecture improves performance across different base model capabilities, with benefits that are not explained by model scale alone. Comparing baseline model performance (4.7–8.2/10) with average agent win rates (72%–98%) shows no inverse correlation. Even the strongest ablation model (GPT OSS 120B, 8.2/10 baseline) shows 97.5% average agent wins across all ablations. This finding shows that agent-based coordination addresses workflow challenges — attention management, validation, information flow — that model scale alone cannot solve. Stronger models benefit from specialization for coordination and completeness rather than compensating for capability gaps.

Organizations should evaluate multi-agent architectures even when using strong base models, budgeting for agent infrastructure from project inception rather than treating it only as an enhancement for

weaker models. The evidence cautions against assuming that upgrading the base model alone eliminates the need for specialization in complex workflows.

5.5.2. Agent utility varies by function: Universal vs conditional

Agents differ in how consistently they help across models. Some provide relatively stable value across the five ablation models, while others show model-dependent effectiveness. The discovery agent shows low variance in win rates (std dev 12.4%), providing consistent benefit in this benchmark. In contrast, the report generation agent shows higher variance (std dev 20.5%), helping some models more than others. This pattern suggests that data discovery is a broadly shared bottleneck in the evaluated setting, while report synthesis depends more heavily on base model generation capabilities.

System designers can use these results as a starting hypothesis: prioritize discovery and analysis agents when building comparable open-data systems, then profile report and intent agents on representative queries before deployment. In resource-constrained environments, a staged deployment can begin with discovery and analysis, adding report or intent agents when profiling shows clear benefit. Multi-model systems may also benefit from separate agent configurations for each model rather than assuming that one architecture is equally efficient for all models.

5.5.3. Agents mitigate distinct failure types: Non-redundant specialization

Each agent addresses a distinct failure mode rather than redundantly solving the same problem, with failures manifesting differently based on which agent is removed. Analysis of judge explanations reveals two failure categories. Removing discovery or analysis agents causes complete task failure (243–280 instances) where the system produces no meaningful output. Removing report or intent agents causes degraded quality — reduced detail or logical inconsistencies — while still generating output. This hierarchy shows that discovery and analysis form critical infrastructure (their absence causes catastrophic failure), while report and intent provide quality enhancement (their absence causes degradation). The distinct failure signatures show that agents are non-redundant — each addresses a specific failure mode that others cannot compensate for.

These failure patterns provide actionable diagnostics when debugging multi-agent systems. Empty or nonsensical outputs point to issues with discovery or analysis agents. Outputs using wrong data sources point to discovery agent problems, while calculation errors point to analysis agent issues. Outputs lacking structure point to report agent

problems, and outputs addressing the wrong question point to intent agent issues. For staged deployment, establish discovery and analysis infrastructure first as the critical path, then add report and intent agents for quality enhancement once the core functionality is stable.

5.5.4. Architectural benefits persist across task complexity

Agent-based architectures provide consistent value across task complexity levels, addressing structural workflow challenges rather than just complex reasoning. Agent win rates remain stable across question difficulty with no systematic variation: analysis (86%–92%), discovery (84%–94%), report (71%–79%), and intent (81%–87%). If agents primarily helped with difficult reasoning, we would expect dramatically higher win rates on hard questions. The stable pattern shows that agents address structural challenges — task decomposition, information flow, output integration — that affect all questions regardless of inherent complexity. This shows that agent benefits stem from workflow management rather than enhanced reasoning capability, guiding future agent design toward coordination mechanisms rather than pure intelligence augmentation.

Organizations should consider agent infrastructure across query types rather than reserving agents only for obviously complex tasks. In this benchmark, simple questions also benefited from the full architecture. If cost reduction is necessary, future work should compare model-selection strategies against selective agent removal, rather than assuming that agents can be safely disabled for easy queries.

5.5.5. Model-agent fit determines effectiveness: Architecture must adapt to model

Optimal agent architecture depends on model-specific strengths and weaknesses, with the same agent providing drastically different value across models. The analysis agent shows 42%–96% win rate variance across models. GPT-4o Mini requires the analysis agent primarily for factual grounding (42% win rate on factual consistency), while GPT OSS 120B benefits uniformly (92%–96% across all criteria). Weaker models need agents for basic functionality (correctness), while stronger models benefit from agents for coordination and completeness. This variance shows that no single agent configuration is optimal across all models—model-specific weaknesses determine which agents provide the most value, and applying uniform agent architectures across different models wastes resources on low-value agents while potentially under-investing in high-value ones.

Before production deployment, system designers should conduct model profiling: evaluate the base model on 20–50 representative queries without agents, categorize weaknesses by failure type (factual errors suggest analysis agent need, missing data suggests discovery agent need, poor structure suggests report agent need, wrong focus suggests intent agent need), run targeted ablations to measure each agent's win rate, then activate agents showing greater than 60% win rates while deferring agents below 40%. For production systems serving multiple models, maintain model-specific agent configurations and implement adaptive routing that activates agents based on runtime model capability assessment for specific query types, balancing quality maintenance with inference cost optimization.

6. Conclusion

We presented PublicAgent, a multi-agent framework for open data analysis, and evaluated how specialized agents contribute to end-to-end report generation from ambiguous public-data questions. Across eight benchmarked models and five-model ablations on 50 benchmark queries, the results suggest five empirical design guidelines: (1) specialization can improve performance across different model strengths, (2) discovery and analysis agents provide more consistent value than report and intent agents, (3) different agents mitigate distinct failure modes, (4) agent benefits are not limited to the hardest questions, and

(5) effective architecture should account for model-specific strengths and weaknesses.

These findings should be interpreted within the scope of the benchmark. The query set covers six public-data domains and provides a realistic test of dataset discovery, schema interpretation, statistical execution, and report synthesis, but it is still limited in size. The evaluation also relies on LLM-based judging, which enables scalable multidimensional assessment but should be complemented by human validation and uncertainty estimates in future work. The added newer-model runs show that capability gains do not automatically translate into uniformly better open-data workflow performance, and continued evaluation will be needed as base-model capabilities change.

Within these limits, PublicAgent demonstrates how natural language interfaces and specialized agent decomposition can reduce technical barriers for journalists, policymakers, and public users seeking evidence from open datasets. Future research should expand the benchmark, add human evaluation subsets, report confidence intervals, compare against general agent frameworks, and develop adaptive routing mechanisms that select agents based on runtime task and model characteristics.

CRedit authorship contribution statement

Sina Montazeri: Writing – original draft, Validation, Methodology, Formal analysis, Conceptualization. **Yunhe Feng:** Writing – review & editing, Validation, Supervision, Project administration, Methodology, Conceptualization. **Kewei Sha:** Writing – review & editing, Validation, Supervision, Project administration, Methodology, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper. This research was supported in part by National Science Foundation (NSF) under Award No. 2505686. The funding source had no involvement in study design, data collection, analysis, interpretation, or the decision to submit the article for publication.

Acknowledgments

This work was supported in part by the National Science Foundation (NSF) under Award No. 2505686.

Data availability

Data will be made available on request.

References

- [1] M. Hilbert, P. López, The world's technological capacity to store, communicate, and compute information, *Science* 332 (6025) (2011) 60–65.
- [2] D. Boyd, K. Crawford, Critical questions for big data: Provocations for a cultural, technological, and scholarly phenomenon, *Inf. Commun. Soc.* 15 (5) (2012) 662–679.
- [3] L. Chen, Y. Deng, Y. Bian, Z. Qin, B. Wu, T.-S. Chua, K.-F. Wong, Beyond factuality: A comprehensive evaluation of large language models as knowledge generators, in: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, Singapore, 2023, pp. 10404–10423.
- [4] G. Faggioli, L. Dietz, C.L.A. Clarke, G. Demartini, M. Hagen, C. Hauff, N. Kando, E. Kanoulas, M. Potthast, B. Stein, H. Wachsmuth, Perspectives on large language models for relevance judgment, in: *Proceedings of the 2023 ACM SIGIR International Conference on Theory of Information Retrieval*, ACM, 2023, pp. 39–50.

- [5] Z. Li, H. Zhu, Z. Lu, M. Yin, Synthetic data generation with large language models for text classification: Potential and limitations, in: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, Singapore, 2023, pp. 10443–10461.
- [6] D.-H. Lee, J. Pujara, M. Sewak, R.W. White, S.K. Jauhar, Making large language models better data creators, in: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, Singapore, 2023, pp. 10188–10207.
- [7] J. Li, G. Li, C. Tao, J. Li, H. Zhang, F. Liu, Z. Jin, Large language model-aware in-context learning for code generation, in: *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Association for Computational Linguistics, 2024, pp. 2140–2152.
- [8] R. Muffoletto, S. Lu, S. Longpre, P. Pasupat, D.S. Weld, M. Friedman, L2CEval: Evaluating language-to-code generation capabilities of large language models, *Trans. Assoc. Comput. Linguist.* 12 (2024) 180–200.
- [9] W.A. Woods, R.M. Kaplan, B. Nash-Webber, The LUNAR Sciences Natural Language Information System: Final Report, Tech. rep., Bolt, Beranek and Newman Inc., Cambridge, MA, 1972.
- [10] X. Xu, C. Liu, D. Song, SQLNet: Generating structured queries from natural language without reinforcement learning, 2017, arXiv preprint [arXiv:1711.04436](https://arxiv.org/abs/1711.04436).
- [11] B. Wang, R. Shin, X. Liu, O. Polozov, M. Richardson, RAT-SQL: Relation-aware schema encoding and linking for text-to-SQL parsers, in: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, Association for Computational Linguistics, 2020, pp. 7567–7578.
- [12] T. Yu, R. Wang, Q. Zhang, Z. Zhang, Y. Yao, S. Liu, Z. Li, R. Zhang, H. Wang, T. Mei, C. Huang, W. Huang, Y. Ma, Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task, in: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, 2018, pp. 3911–3921.
- [13] V. Zhong, C. Xiong, R. Socher, Seq2SQL: Generating structured queries from natural language using reinforcement learning, 2017, ArXiv Preprint [arXiv:1709.00103](https://arxiv.org/abs/1709.00103).
- [14] J. Li, B. Hui, G. Qu, B. Yang, J. Li, B. Li, B. Wang, B. Qin, R. Geng, N. Huo, et al., BIRD: A big bench for large-scale database grounded text-to-SQL evaluation, 2023, arXiv preprint [arXiv:2305.03111](https://arxiv.org/abs/2305.03111).
- [15] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, J. Zhou, DAIL-SQL: Text-to-SQL empowered by large language models: A benchmark evaluation, 2023, arXiv preprint [arXiv:2308.15363](https://arxiv.org/abs/2308.15363).
- [16] L. Kothhoff, C. Thornton, H.H. Hoos, F. Hutter, K. Leyton-Brown, Auto-WEKA 2.0: Automatic model selection and hyperparameter optimization in WEKA, *J. Mach. Learn. Res.* 18 (1) (2017) 826–830.
- [17] M. Feurer, A. Klein, K. Eggenberger, J. Springenberg, M. Blum, F. Hutter, Efficient and robust automated machine learning, in: *Advances in Neural Information Processing Systems*, 2015, pp. 2962–2970.
- [18] R.S. Olson, J.H. Moore, TPOT: A tree-based pipeline optimization tool for automating machine learning, in: *Workshop on Automatic Machine Learning*, PMLR, 2016, pp. 66–74.
- [19] J. Kaddour, J. Harris, M. Mozes, H. Bradley, R. Raileanu, R. McHardy, AIDE: An automatic data engine for object detection in autonomous driving, 2024, arXiv preprint [arXiv:2403.17373](https://arxiv.org/abs/2403.17373).
- [20] C. Wang, E. Hoque, B. Lim, S.M. Drucker, Data Formulator 2: Iteratively creating rich visualizations with AI, 2024, arXiv preprint [arXiv:2408.16119](https://arxiv.org/abs/2408.16119).
- [21] V. Dibia, LIDA: A tool for automatic generation of grammar-agnostic visualizations and infographics using large language models, in: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, 2023, pp. 113–126.
- [22] Y. Han, C. Zhang, X. Chen, X. Yang, Z. Wang, G. Yu, B. Fu, H. Zhang, ChartLlama: A multimodal LLM for chart understanding and generation, 2023, arXiv preprint [arXiv:2311.16483](https://arxiv.org/abs/2311.16483).
- [23] S. Hong, X. Zheng, J. Chen, Y. Cheng, J. Zhang, C. Wang, Z. Wang, S.K.S. Yau, Z. Lin, L. Zhou, et al., MetaGPT: Meta programming for multi-agent collaborative framework, 2023, arXiv preprint [arXiv:2308.00352](https://arxiv.org/abs/2308.00352).
- [24] Q. Wu, G. Bansal, J. Zhang, Y. Wu, S. Zhang, E. Zhu, B. Li, L. Jiang, X. Zhang, C. Wang, AutoGen: Enabling next-gen LLM applications via multi-agent conversation framework, 2023, arXiv preprint [arXiv:2308.08155](https://arxiv.org/abs/2308.08155).
- [25] B. Qiao, L. Gong, S. Prasad, X. Ren, P. Cui, K. Eggenberger, X. Zhang, M. Li, L. Li, Z. Zhou, et al., TaskWeaver: A code-first agent framework for seamlessly planning and executing data analytics tasks, 2023, arXiv preprint [arXiv:2311.17541](https://arxiv.org/abs/2311.17541).
- [26] Y. Shao, Y. Zhang, Z. Gu, X.V. Lin, Co-STORM: Collaborative knowledge curation with large language models, 2024, arXiv preprint [arXiv:2408.15232](https://arxiv.org/abs/2408.15232).
- [27] C. Qian, X. Liu, H. Fu, Y. Meng, Z. Wang, X. Wang, W. Xu, Y. Chen, J. Zhang, Y. Huang, et al., ChatDev: Communicative agents for software development, in: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024, pp. 15174–15186.
- [28] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, Y. Zhuang, HuggingGPT: Solving AI tasks with ChatGPT and its friends in hugging face, 2023, arXiv preprint [arXiv:2303.17580](https://arxiv.org/abs/2303.17580).
- [29] Q. Wang, T. Wang, Z. Tang, Q. Li, N. Chen, J. Liang, B. He, MegaAgent: A large-scale autonomous LLM-based multi-agent system without predefined SOPs, in: *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 4998–5036, [arXiv:2408.09955](https://arxiv.org/abs/2408.09955).
- [30] W. Chen, Y. Su, J. Zuo, C. Yang, C. Yuan, C.-M. Chan, H. Yu, Y. Lu, Y.-H. Hung, C. Qian, et al., AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors in agents, 2023, arXiv preprint [arXiv:2308.10848](https://arxiv.org/abs/2308.10848).
- [31] A. Li, Y. Xie, S. Li, F. Tsung, B. Ding, Y. Li, Agent-oriented planning in multi-agent systems, 2024, arXiv preprint [arXiv:2410.02189](https://arxiv.org/abs/2410.02189), Accepted by ICLR 2025.
- [32] K.-T. Tran, D. Dao, M.-D. Nguyen, et al., Multi-agent collaboration mechanisms: A survey of LLMs, 2025, arXiv preprint [arXiv:2501.06322](https://arxiv.org/abs/2501.06322).
- [33] S. Han, Q. Zhang, Y. Yao, W. Jin, Z. Xu, LLM multi-agent systems: Challenges and open problems, 2024, arXiv preprint [arXiv:2402.03578](https://arxiv.org/abs/2402.03578).
- [34] Anthropic, Claude Code documentation, 2025, URL <https://docs.anthropic.com/en/docs/claude-code/overview>, Online documentation.
- [35] Google, Gemini CLI, 2025, URL <https://github.com/google-gemini/gemini-cli>, GitHub repository.
- [36] OpenClaw, OpenClaw ACP Agents, 2026, URL <https://openclaws.io/docs/tools/acp-agents/>, Online documentation.
- [37] H. Zamani, S. Dumais, N. Craswell, P. Bennett, G. Lueck, Generating clarifying questions for information retrieval, 2020, arXiv preprint [arXiv:2005.03736](https://arxiv.org/abs/2005.03736).
- [38] M. Allannejadi, H. Zamani, F. Crestani, W.B. Croft, ConvaI3: Generating clarifying questions for open-domain dialogue systems (ClariQ), 2020, ArXiv Preprint [arXiv:2009.11352](https://arxiv.org/abs/2009.11352).
- [39] F. Li, H.V. Jagadish, Constructing an interactive natural language interface for relational databases, *Proc. the VLDB Endow.* 8 (1) (2014) 73–84.
- [40] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, F. Wei, Query2doc: Query expansion with large language models, 2023, arXiv preprint [arXiv:2303.07678](https://arxiv.org/abs/2303.07678).
- [41] B. Wu, L. He, L. Yang, F. Wang, Y. Li, CoT-BERT: Enhancing unsupervised sentence representation through chain-of-thought, 2023, arXiv preprint [arXiv:2309.11143](https://arxiv.org/abs/2309.11143).
- [42] E. Goldberg, N. Driedger, R.I. Kittredge, Using natural language processing to produce weather forecasts, *IEEE Expert* 9 (2) (1994) 45–53.
- [43] E. Reiter, S. Sripada, J. Hunter, J. Yu, I. Davy, Choosing words in computer-generated weather forecasts, *Artificial Intelligence* 167 (1–2) (2005) 137–169.
- [44] J. Herzig, P.K. Nowak, T. Müller, F. Piccinno, J. Eisenschlos, TaPas: Weakly supervised table parsing via pre-training, in: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020, pp. 4320–4333.
- [45] Z. Chen, W. Chen, C. Smiley, S. Shah, I. Borova, D. Langdon, R. Moussa, M. Beane, T.-H. Huang, B. Routledge, et al., FinQA: A dataset of numerical reasoning over financial data, in: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021, pp. 3697–3711.
- [46] R. Allen, M. Cafarella, S. Gopalakrishnan, K. Hammond, M.A. Walker, A. Raphaely, K. Ramakrishnan, K. Sarver, J. Sedoc, Y. Stylianou, StatsMonkey: A data-driven sports narrative writer, *Comput. Linguist. Context.* (2010).
- [47] A. Gatt, E. Krahmer, Survey of the state of the art in natural language generation: Core tasks, applications and evaluation, *J. Artificial Intelligence Res.* 61 (2018) 65–170.
- [48] LangChain, LangGraph: Building stateful, multi-actor applications with LLMs, 2024, URL <https://langchain-ai.github.io/langgraph/>, Software documentation.
- [49] N.F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, P. Liang, Lost in the middle: How language models use long contexts, *Trans. Assoc. Comput. Linguist.* 12 (2023) 157–173, URL <https://api.semanticscholar.org/CorpusID:259360665>.
- [50] C.-Y. Hsieh, C.-L. Li, C.-K. Yeh, H. Nakhost, Y. Fujii, A. Ratner, R. Krishna, C.-Y. Lee, T. Pfister, RULER: What's the real context size of your long-context language model?, 2024, [arXiv:2404.06611](https://arxiv.org/abs/2404.06611).
- [51] X. Liu, Y. Xu, Z. Wu, T. Wang, Y. Chen, D. Zhao, R. Yan, BIRD: A large-scale, diversified, and realistic dataset for text-to-SQL, in: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (Volume 1: Long Papers)*, Association for Computational Linguistics, 2023, pp. 6028–6041.
- [52] C.-H. Lee, O. Polozov, M. Richardson, Kaggledbqa: Realistic evaluation of text-to-SQL parsers, in: *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, Association for Computational Linguistics, Online, 2021, pp. 2261–2273, <http://dx.doi.org/10.18653/v1/2021.acl-long.176>.
- [53] M.T. Ribeiro, T. Wu, C. Guestrin, S. Singh, Beyond accuracy: Behavioral testing of NLP models with CheckList, in: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, Association for Computational Linguistics, Online, 2020, pp. 4902–4912, <http://dx.doi.org/10.18653/v1/2020.acl-main.442>.
- [54] A. Holtzman, J. Buys, L. Du, M. Forbes, Y. Choi, The curious case of neural text degeneration, in: *International Conference on Learning Representations*, 2020.
- [55] M. Renze, E. Guven, The effect of sampling temperature on problem solving in large language models, in: *Findings of the Association for Computational Linguistics: EMNLP 2024*, Association for Computational Linguistics, Miami, Florida, USA, 2024, pp. 7379–7393.

- [56] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C.L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askeel, P. Welinder, P. Christiano, J. Leike, R. Lowe, Training language models to follow instructions with human feedback, in: *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 27730–27744.
- [57] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing, Y. Katsis, D. Zhang, H. Cheng, Y. Zhang, Y. Wang, et al., Judging LLM-as-a-judge with MT-bench and chatbot arena, 2023, [arXiv:2306.05685](#).
- [58] X. Lin, G.I. Winata, P. Xu, Yoplop, H.M. Thai, C.-C. Chang, S. Qian, M. Liu, W. Zhang, F.F. Xu, Z. Niu, R. Xu, P. Stenetorp, P. Fung, AlpacaEval 2.0: An evolving LLM-as-a-judge evaluation, 2024, [arXiv:2402.00928](#).
- [59] J.S. Park, J.C. O'Brien, C.J. Cai, M.R. Morris, P. Liang, M.S. Bernstein, Prometheus: Inducing fine-grained evaluation capability in language models, 2024, [arXiv:2310.08491](#).
- [60] Z. Shen, K. Long, H. Gao, S. Wang, Y. Gu, H. Zhang, S. Shi, S. Wang, S. Wang, et al., LLM-as-a-judge: A comprehensive survey, 2024, [arXiv:2405.14387](#).
- [61] S. Cho, S. Yang, Y. Wang, L. Zettlemoyer, D. Fried, H. Hajishirzi, N.A. Smith, LLM-as-a-judge: Biases and calibration, 2024, [arXiv:2404.00031](#).
- [62] Y. Dubois, B. Galambosi, P. Liang, T.B. Hashimoto, Length-controlled AlpacaEval: A simple way to debias automatic evaluators, in: *First Conference on Language Modeling*, 2024.
- [63] R. Meyers, M. Lu, C. Wabert de Puiseau, T. Meisen, Ablation studies in artificial neural networks, 2019, arXiv preprint [arXiv:1901.08644](#).